# Vilvah – Advanced SQL Query System

## Introduction

Vilvah is a DBMS project based on a natural skincare and personal care e-commerce business. It uses SQL queries to manage and analyze customer, product, order, and sales information. Advanced SQL concepts help retrieve meaningful information, compare values, and generate useful business reports.

## Objectives

1. Implement subqueries and nested queries.
2. Find products above and below the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

## 1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query. The inner query executes first, and its result is used by the outer query.

## Types of subqueries:

1. Single-row subquery – Returns one value.
2. Multiple-row subquery – Returns multiple values.
3. Nested subquery – A subquery inside another subquery.
4. Correlated subquery – The inner query depends on the outer query.

**Example:** Find orders whose amount is greater than the average order amount.

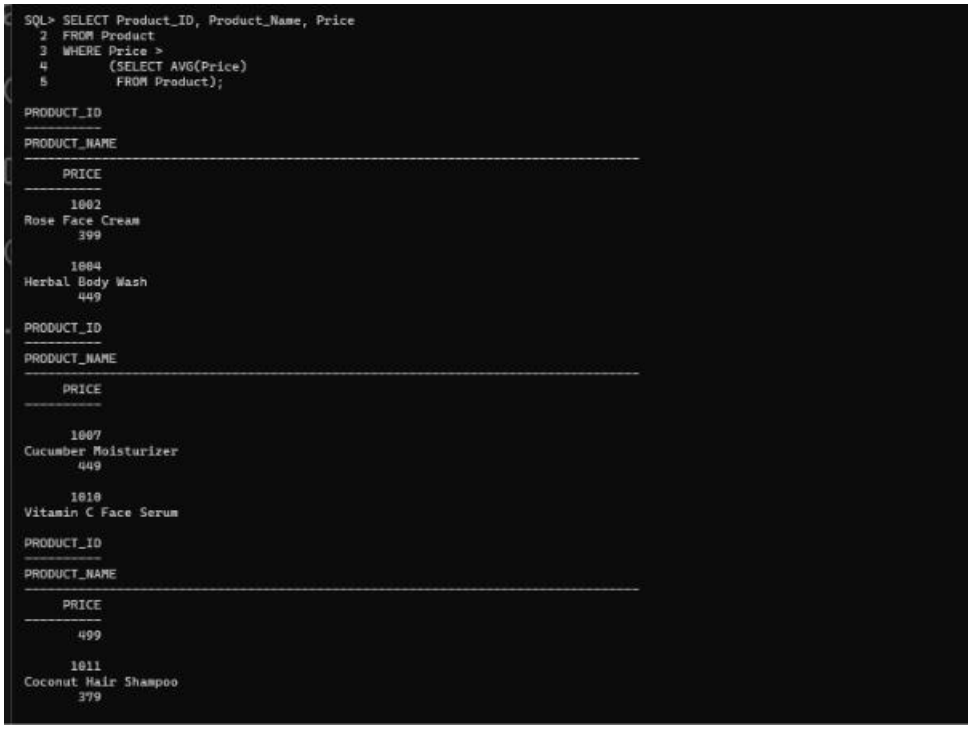
```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
  2  FROM Orders
  3  WHERE Total_Amount >
  4        (SELECT AVG(Total_Amount)
  5         FROM Orders);

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
       101           1          798
       103           3          999
       106           6         1200
       108           8          899
       110          10         1100
       111           1          850
       120          10          899

7 rows selected.
```

## 2. Find Products Above Average Price

This query identifies products whose prices are greater than the average price of all products.



```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price >
  4        (SELECT AVG(Price)
  5         FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
      1002
Rose Face Cream
       399

      1004
Herbal Body Wash
       449

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

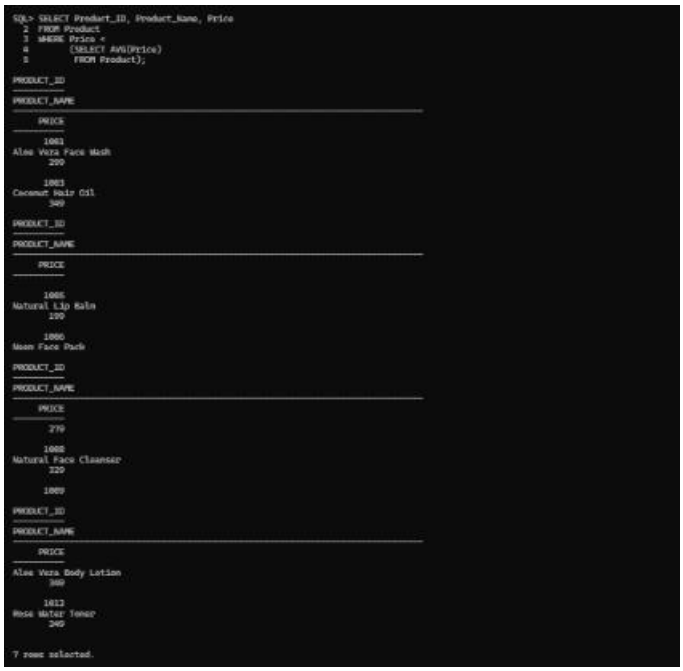
      1007
Cucumber Moisturizer
       449

      1010
Vitamin C Face Serum

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       499

      1011
Coconut Hair Shampoo
       379
```

## 2.1 Products Below Average Price



```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price <
  4        (SELECT AVG(Price)
  5         FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
      1001
Aloe Vera Face Wash
       299

      1003
Coconut Hair Oil
       349

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

      1005
Natural Lip Balm
       199

      1006
Neem Face Pack

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       270

      1008
Natural Face Cleanser
       329

      1009

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
Aloe Vera Body Lotion
       389

      1013
Rose Water Toner
       249

7 rows selected.
```

## 3. Identify Customers with Maximum Purchases

This query uses SUM() and a subquery to find the customer with the highest total purchase amount.

## Step 1: Calculate customer-wise purchases.

```
SQL> SELECT Customer_ID,
  2         SUM(Total_Amount) AS Total_Purchase
  3  FROM Orders
  4  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          1           1648
          2            599
          3            999
          4            450
          5            750
          6           1749
          7            650
          8           1298
          9           1199
         10           1999

10 rows selected.
```

## Step 2: Find the maximum purchase.

```
SQL> SELECT MAX(Total_Purchase)
  2  FROM (
  3      SELECT Customer_ID,
  4             SUM(Total_Amount) AS Total_Purchase
  5      FROM Orders
  6      GROUP BY Customer_ID
  7  );

MAX(TOTAL_PURCHASE)
-------------------
               1999
```

## 3.1 Display Customer Details with Maximum Purchase

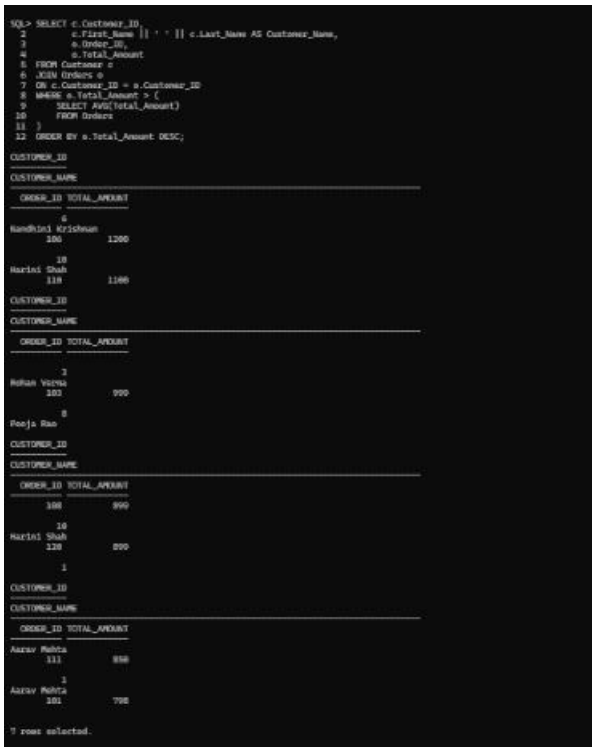
```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         SUM(o.Total_Amount) AS Total_Purchase
  4  FROM Customer c
  5  JOIN Orders o
  6  ON c.Customer_ID = o.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  HAVING SUM(o.Total_Amount) = (
  9      SELECT MAX(Total_Purchase)
 10      FROM (
 11          SELECT SUM(Total_Amount) AS Total_Purchase
 12          FROM Orders
 13          GROUP BY Customer_ID
 14      )
 15  );

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
         10
Harini Shah
          1999
```

## 4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

## 4.1 Customers Who Placed Orders Above Average Order Value



```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         o.Order_ID,
  4         o.Total_Amount
  5  FROM Customer c
  6  JOIN Orders o
  7  ON c.Customer_ID = o.Customer_ID
  8  WHERE o.Total_Amount > (
  9      SELECT AVG(Total_Amount)
 10      FROM Orders
 11  )
 12  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
         6
Nandhini Krishnan
       106         1200

        10
Harini Shah
       110         1100

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------

         3
Rohan Verma
       103          999

         8
Pooja Rao

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
       108          999

        10
Harini Shah
       120          899

         1

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
Aarav Mehta
       111          850

         1
Aarav Mehta
       101          700

7 rows selected.
```

## 5. Find Products Whose Sales Are Above Average

This query identifies products whose total sales are greater than the average sales generated by all products.

```
SQL> SELECT p.Product_ID,
  2         p.Product_Name,
  3         SUM(od.Total_Price) AS Total_Sales
  4  FROM Product p
  5  JOIN Order_Details od
  6  ON p.Product_ID = od.Product_ID
  7  GROUP BY p.Product_ID, p.Product_Name
  8  HAVING SUM(od.Total_Price) > (
  9      SELECT AVG(Product_Sales)
 10      FROM (
 11          SELECT SUM(Total_Price) AS Product_Sales
 12          FROM Order_Details
 13          GROUP BY Product_ID
 14      )
 15  )
 16  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
      1009
Aloe Vera Body Lotion
      1248

      1006
Neem Face Pack
      1200

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------

      1010
Vitamin C Face Serum
      1100

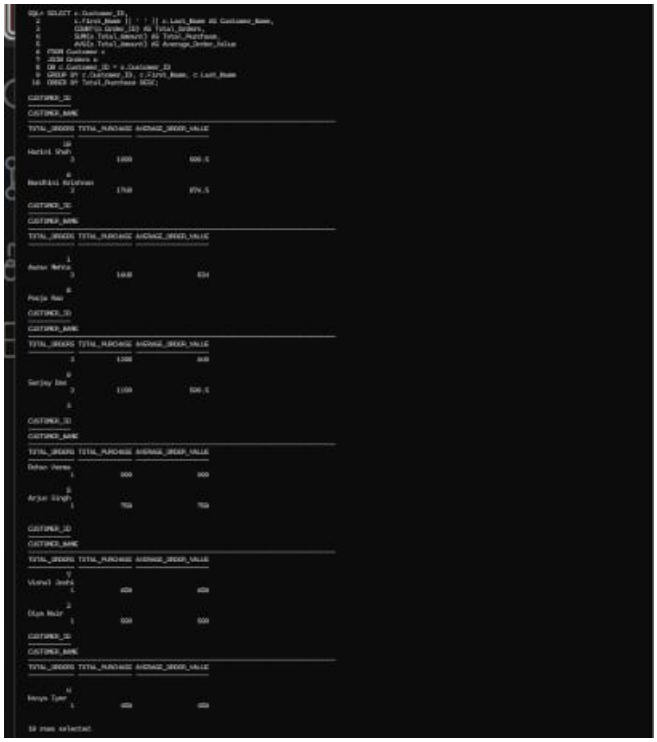
      1003
Coconut Hair Oil

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
       999

      1008
Natural Face Cleanser
       800
```

## 6. Customer Purchase Report

This report combines Customer and Orders tables to display customer purchase details.



## Report contains:

1.Customer ID

2.Customer name

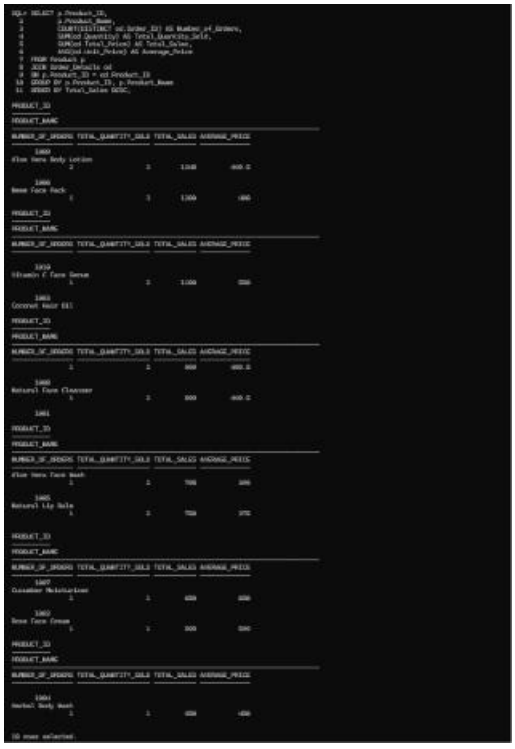
3.Number of orders

4.Total purchase amount

5.Average order value

## 7. Advanced Product Sales Report

This report combines Product and Order_Details tables to analyze product performance.



**Purpose:** This report helps identify the number of orders, total quantity sold, total sales, average selling price, and products generating higher sales.

## 8. Category-wise Advanced Sales Report

This report combines Category, Product, and Order_Details tables to analyze sales by product category.

```
SQL> SELECT cat.Category_ID,
  2         cat.Category_Name,
  3         COUNT(DISTINCT od.Order_ID) AS Total_Orders,
  4         SUM(od.Quantity) AS Total_Quantity_Sold,
  5         SUM(od.Total_Price) AS Total_Sales,
  6         AVG(od.Unit_Price) AS Average_Price
  7  FROM Category cat
  8  JOIN Product p
  9  ON cat.Category_ID = p.Category_ID
 10  JOIN Order_Details od
 11  ON p.Product_ID = od.Product_ID
 12  GROUP BY cat.Category_ID, cat.Category_Name
 13  ORDER BY Total_Sales DESC;

CATEGORY_ID CATEGORY_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
------------------- ----------- -------------
          1 Face Care                                                     3
                  5        2497           516

          2 Body Care                                                     3
                  4        1698    449.666667

          3 Hair Care                                                     1
                  2         999         499.5
```